

Handbook Module 2

Artificial Intelligence

Module 2: Mathematical Foundation and Data Visualization for AI

Written By: Zahran Alvan Putra Winarko, GDGOC ITB Curriculum Team

Linear Algebra in the ML Context

Vectors as Data Representations

In machine learning, every data point is represented as a vector – an ordered collection of numbers in one dimension. If we have a car dataset with features: fuel efficiency, weight, engine power, and acceleration, then a single car can be represented as:

$$\mathbf{x} = \begin{pmatrix} 23.0 \\ 2970 \\ 98 \\ 15.5 \end{pmatrix} \leftarrow \begin{pmatrix} \text{mpg} \\ \text{weight (lbs)} \\ \text{horsepower} \\ \text{acceleration} \end{pmatrix}$$

This is the feature vector of a single data point. An entire dataset with n rows and d features then becomes a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ – where each row is one data point and each column is one feature.

Vector Operations That Appear Frequently in ML

1. Dot Product

The dot product (or inner product) between two vectors $\mathbf{a}$ and $\mathbf{b}$ is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

In ML, the dot product appears most commonly in three forms:

- Linear prediction:** $\hat{y} = \mathbf{w} \cdot \mathbf{x} = \sum_i w_i x_i$ – the $\mathbf{w}$ and the input feature vector $\mathbf{x}$.
- Cosine similarity:** $\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$ – a measure of directional similarity between two vectors, already used in the Module 1 bonus task.
- Attention mechanism:** transformers use the dot product between query and key vectors to determine how much attention to assign to each token in the sequence.

2. Vector Norms: L1 and L2

A vector norm measures the “length” or magnitude of a vector – intuitively, the distance from the origin to the point represented by that vector.

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} \quad (\text{Euclidean / L2 norm})$$

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| \quad (\text{Manhattan / L1 norm})$$

Relevance in ML: when a model is penalised to keep its weights from growing too large, we add a norm of the weights to the loss function. L2 regularisation (Ridge) adds $\lambda \|\mathbf{w}\|_2^2$, pushing all weights to shrink proportionally. L1 regularisation (Lasso) adds $\lambda \|\mathbf{w}\|_1$, pushing some weights to exactly zero and thereby performing automatic feature selection.

```
import numpy as np

a = np.array([3.0, 4.0, 0.0, 1.0])

# Dot product
b = np.array([1.0, 2.0, 3.0, 0.5])
print("Dot product:", np.dot(a, b))           # 3*1 + 4*2 +
0*3 + 1*0.5 = 11.5

# L2 norm
print("L2 norm:", np.linalg.norm(a))          #
sqrt(9+16+0+1) = 5.099
print("L2 norm (manual):", np.sqrt((a**2).sum()))

# L1 norm
print("L1 norm:", np.linalg.norm(a, ord=1))   # 3+4+0+1 = 8.0

# Cosine similarity from scratch (same as Module 1 bonus)
cos_sim = np.dot(a, b) / (np.linalg.norm(a) *
np.linalg.norm(b))
print("Cosine similarity:", round(cos_sim, 4))
```

Matrices and Matrix Multiplication

An entire ML dataset is a matrix. If we have 398 cars each described by 5 numeric features, the dataset is $\mathbf{X} \in \mathbb{R}^{398 \times 5}$.

Matrix multiplication $C = AB$ (where $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, giving $C \in \mathbb{R}^{m \times n}$) appears everywhere in ML:

- **Neural network forward pass:** $h = XW^T + b$ is a single affine transformation layer.
- **Normal equation for linear regression:** the optimal weights are obtained from $\hat{\theta} = (X^T X)^{-1} X^T y$.
- **Covariance matrix:** $\Sigma = \frac{1}{n-1} X^T X$ (after mean-centering) – the foundation of PCA.

```
import numpy as np

# Mini simulation: 3 data points, 2 features
X = np.array([[1, 2],
               [3, 4],
               [5, 6]])

# Linear model weights
W = np.array([0.5, -0.3])

# Predict all data points at once via matrix-vector
multiplication
y_pred = X @ W      # same as np.dot(X, W)
print("Predictions:", y_pred)  # [1*0.5+2*(-0.3), ...] =
[-0.1, 0.3, 0.7]

# Transpose: X.T has shape (2, 3)
print("X.T shape:", X.T.shape)

# Gram matrix: X.T @ X -- appears in the normal equation
gram = X.T @ X
print("Gram matrix:\n", gram)
```

References and Further Reading

[Essence of Linear Algebra](#)
[Linear Algebra Review and Reference](#)
[NumPy Linear Algebra](#)

Statistics for ML

Descriptive Statistics – Contextualised to Training

Descriptive statistics are not just about "describing data" – in ML, every number has a direct implication for model behaviour.

1. Mean and Variance

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2 \quad \sigma = \sqrt{\sigma^2}$$

ML implication: when features have very different means and variances across columns (e.g. weight in the thousands vs mpg in the tens), gradient-descent-based models struggle because the loss landscape is tilted – gradients along the high-scale feature direction dominate and updates are inefficient. This is why feature scaling is not optional – it is almost always mandatory before training.

$$x_{\text{normalized}} = \frac{x - \min}{\max - \min} \quad (\text{min-max}) \quad x_{\text{standardized}} = \frac{x - \mu}{\sigma} \quad (\text{z-score})$$

2. Median and IQR – Robust to Outliers

Mean and variance are sensitive to outliers. A single extreme value can shift the mean significantly. Median (the middle value after sorting) and IQR (Q3 minus Q1) are far more robust:

```
import numpy as np

data = np.array([10, 12, 11, 13, 14, 10, 11, 500]) # one
extreme outlier

print(f"Mean: {data.mean():.1f}")                # 72.6 --
heavily skewed by outlier
print(f"Median: {np.median(data):.1f}")           # 11.5 --
nearly unchanged

q1, q3 = np.percentile(data, [25, 75])
iqr = q3 - q1
print(f"Std: {data.std():.1f}, IQR: {iqr:.1f}")    # IQR is
far smaller than std
```

Normal Distribution and Why It Matters in ML

The normal (Gaussian) distribution appears so frequently in ML not by accident but because of the **Central Limit Theorem (CLT)**: the mean of a sufficiently large sample from any distribution will approach a normal distribution, regardless of the original distribution's shape.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

The 68-95-99.7 rule: in a normal distribution,

- ~68% of data falls within $[\mu - \sigma, \mu + \sigma]$
- ~95% of data falls within $[\mu - 2\sigma, \mu + 2\sigma]$
- ~99.7% of data falls within $[\mu - 3\sigma, \mu + 3\sigma]$

Why it matters in ML:

- Many models assume that residuals (errors between predictions and actual values) are normally distributed – this assumption underlies linear regression.
- Neural network weight initialisation often uses a normal distribution (e.g. Xavier initialisation uses $\mathcal{N}(0, 1/\text{fan_in})$).
- Z-score standardisation uses μ and σ – and it is most meaningful when the original data is approximately normal.

```
import numpy as np

rng = np.random.default_rng(42)
data = rng.normal(loc=0, scale=1, size=10000) # normal
dist: mu=0, sigma=1

mu, sigma = data.mean(), data.std()
within_1std = np.abs(data - mu) <= sigma
within_2std = np.abs(data - mu) <= 2*sigma

print(f"Within 1 std: {within_1std.mean()*100:.1f}%") #
~68%
print(f"Within 2 std: {within_2std.mean()*100:.1f}%") #
~95%
```

Covariance measures how much two variables move together. Pearson correlation is the normalised version of covariance, so its value always lies between -1 and +1:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu_X)(y_i - \mu_Y)$$
$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Interpretation:

- $r \approx +1$: strong positive correlation (one goes up, so does the other)
- $r \approx -1$: strong negative correlation (one goes up, the other goes down)
- $r \approx 0$: no linear correlation (but nonlinear correlation may still exist!)

Important: high correlation between two input features (multicollinearity) can be problematic for some models because they carry redundant information.

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 5, 4, 5], dtype=float)

# Pearson correlation from scratch
cov_xy = np.mean((x - x.mean()) * (y - y.mean()))
r = cov_xy / (x.std() * y.std())
print(f"Pearson correlation (manual): {r:.4f}")

# Verify using np.corrcoef
corr_matrix = np.corrcoef(x, y)
print(f"Correlation via np.corrcoef: {corr_matrix[0,1]:.4f}")
```

Bayes' Theorem – A Teaser

Bayes' theorem describes how we update our beliefs in the light of new evidence:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

This forms the foundation of the **Naive Bayes classifier** (Module 4) and more broadly of Bayesian inference in probabilistic ML. The intuition is simple: *prior* is

the belief before seeing data, *likelihood* is how probable the data would be if our belief were correct, and *posterior* is the updated belief after seeing the data.

References and Further Readings

[Statistics and Probability](#)
[Seeing Theory](#)
[Bayes' Theorem Explained](#)

Exploratory Data Analysis (EDA)

Why EDA Before Modelling?

EDA is not a formality – it is the stage where you actually *understand* the data before handing it to a model. A model trained without EDA is trained blindly: you do not know whether the data is corrupt, whether features are irrelevant, whether distributions are extreme, or whether key assumptions are violated. Garbage in, garbage out – and EDA is your first line of defence.

Concretely, EDA answers the following questions:

1. Are there missing values, and is the pattern of missingness informative or random?
2. What is the shape of each feature's distribution -- normal, skewed, bimodal?
3. Are there outliers that are genuine data-entry errors vs valid but rare values?
4. Which features correlate strongly with the target? Which features correlate with each other?
5. Are there clear patterns by category (e.g. country of origin, year, class)?

Distribution Analysis

The distribution of a feature can reveal a great deal:

- **Skewness:** a right-skewed distribution (long tail to the right) means a few values are much larger than the majority – mean will be greater than median. Features like income, house prices, or transaction counts are often right-skewed. Log transformation is commonly used to “flatten” such distributions before feeding them into a model.
- **Bimodal:** a histogram with two distinct peaks often signals that the data actually comes from two different populations mixed together – for

example, car data mixing economy cars and sports cars. This can indicate that feature engineering or data segmentation is needed.

- **Uniform:** a flat distribution – nearly equal frequency across all values. This can sometimes mean data was artificially generated or there is a collection issue.

Outlier Detection Using IQR

The IQR method defines outliers as values that fall outside:

$$[Q_1 - 1.5 \times \text{IQR}, \quad Q_3 + 1.5 \times \text{IQR}]$$

This is more robust than the “3 sigma” method that uses mean and std, because mean and std are themselves affected by outliers.

```
import numpy as np

def detect_outliers_iqr(arr: np.ndarray) → np.ndarray:
    q1, q3 = np.percentile(arr, [25, 75])
    iqr = q3 - q1
    lo = q1 - 1.5 * iqr
    hi = q3 + 1.5 * iqr
    return (arr < lo) | (arr > hi)

# Example with horsepower from the MPG dataset
import pandas as pd, seaborn as sns
df = sns.load_dataset('mpg').dropna()
hp = df['horsepower'].to_numpy()
mask = detect_outliers_iqr(hp)
print(f"Horsepower outliers: {mask.sum()} out of {len(hp)} ({mask.mean()*100:.1f}%)")
print(f"Outlier values: {sorted(hp[mask])}")
```

Outliers Are Not Always Noise

Do not remove every outlier automatically. There are two types: (1) **data errors** – values that are impossible given the domain (negative age, horsepower of 1000 for an ordinary car) – these should be removed or imputed. (2) **valid extreme values** – a sports car with 230 hp genuinely exists and should be kept. The decision must be based on domain knowledge, not mechanical rules.

Visualisation with Matplotlib & Seaborn

Matplotlib Figure Anatomy

Before using Seaborn, it is important to understand the basic structure of Matplotlib, since Seaborn is built on top of it:

```
import matplotlib.pyplot as plt
import numpy as np

# Figure: the overall "canvas"
# Axes: a single plot panel inside the figure
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

x = np.linspace(0, 2*np.pi, 100)

# First axes
axes[0].plot(x, np.sin(x), color='steelblue', linewidth=2,
label='sin(x)')
axes[0].set_title('Sine Wave')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Second axes
axes[1].plot(x, np.cos(x), color='coral', linewidth=2,
label='cos(x)')
axes[1].set_title('Cosine Wave')
axes[1].set_xlabel('x')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('trig_waves.png', dpi=150, bbox_inches='tight')
plt.show()
```

Choosing the Right Chart

Plot Type	When to Use	Seaborn Function
Histogram	Distribution of one numeric	<code>sns.histplot()</code>

	variable	
Box plot	Distribution + outliers, comparison across groups	<code>sns.boxplot()</code>
Scatter plot	Relationship between two numeric variables	<code>sns.scatterplot()</code>
Line plot	Trends over time or sequential data	<code>sns.lineplot()</code>
Bar plot	Comparing values across categories	<code>sns.barplot()</code>
Heatmap	Matrix of values (correlation, confusion matrix)	<code>sns.heatmap()</code>
Pair plot	All pairwise feature combinations at once	<code>sns.pairplot()</code>
Violin plot	Distribution + density, more detailed than box	<code>sns.violinplot()</code>

Seaborn Statistical Plots – Practical Examples

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = sns.load_dataset('mpg').dropna()

# --- 1. Histogram: mpg distribution ---
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(df['mpg'], bins=30, kde=True, ax=ax,
color='steelblue')
ax.set_title('Distribution of Fuel Efficiency (mpg)')
ax.set_xlabel('Miles per Gallon')
ax.axvline(df['mpg'].mean(), color='red', linestyle='--',
label=f"Mean: {df['mpg'].mean():.1f}")
ax.axvline(df['mpg'].median(), color='orange',
linestyle='--',
```

```

        label=f"Median: {df['mpg'].median():.1f}")
ax.legend()
plt.tight_layout()
plt.show()

# --- 2. Boxplot per origin ---
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=df, x='origin', y='mpg', palette='Set2',
            ax=ax,
            order=['usa', 'europe', 'japan'])
ax.set_title('Fuel Efficiency by Country of Origin')
ax.set_xlabel('Origin')
ax.set_ylabel('Miles per Gallon')
plt.tight_layout()
plt.show()

# --- 3. Scatter + regression trend line: weight vs mpg ---
fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(data=df, x='weight', y='mpg', hue='origin',
                palette='Set1', alpha=0.7, ax=ax)
z = np.polyfit(df['weight'], df['mpg'], 1)
p = np.poly1d(z)
xline = np.linspace(df['weight'].min(), df['weight'].max(),
                    100)
ax.plot(xline, p(xline), 'k--', linewidth=1.5,
        label='Trend')
ax.set_title(f"Weight vs MPG (r = {df['weight'].corr(df['mpg']):.3f})")
ax.set_xlabel('Weight (lbs)')
ax.set_ylabel('Miles per Gallon')
ax.legend()
plt.tight_layout()
plt.show()

# --- 4. Correlation heatmap ---
numeric_cols =
['mpg', 'displacement', 'horsepower', 'weight', 'acceleration']
corr_matrix = df[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr_matrix, annot=True, fmt='.2f',
            cmap='coolwarm',

```

```
center=0, square=True, ax=ax)
ax.set_title('Feature Correlation Matrix')
plt.tight_layout()
plt.show()
```

Good Visualisation Principles

1. **Every plot must be self-contained** – always include a title, axis labels, and a legend so the chart can be understood without additional explanation.
2. **Choose honest scales** – a Y axis that does not start at zero on a bar chart can make a small difference look dramatic.
3. **Avoid pie charts** for more than 3 categories – the human brain is poor at comparing angles. Use a bar chart instead.
4. **Use colour to encode information, not as decoration** – use hue to distinguish categories, not to make a chart look colourful.
5. **One plot, one question** – if a single chart is trying to answer too many questions at once, that is a signal to split it into multiple subplots.

References and Further Reading

[Seaborn Tutorial](#)
[Matplotlib Tutorials](#)
[From Data to Viz](#)
[Fundamentals of Data Visualization](#)